

PROJETO DE PROGRAMAÇÃO ORIENTADA A OBJETOS

GESTÃO DE OBRAS DE CONSTRUÇÃO CIVIL

NUNO MIGUEL FERNANDES OLIVEIRA – 31550

ORIENTAÇÃO
ERNESTO CASANOVA

ANO LETIVO 2025/2026

INSTITUTO POLITÉCNICO DO CÁVADO E DO AVE
ESCOLA SUPERIOR DE TECNOLOGIA
ENGENHARIA DE SISTEMAS INFORMÁTICOS

Índice

Índice	2
Índice de Figuras	3
Índice de Tabelas	3
1. Introdução	4
2. Objetivo	4
3. Análise do problema	4
4. Levantamento de Requisitos	4
4.1. Requisitos Funcionais	4
4.2. Requisitos Não Funcionais	5
5. Identificação das Principais Classes	5
5.1. Obra	5
5.1.1. Propriedades	5
5.1.2. Métodos	6
5.2. Material	6
5.2.1. Propriedades	6
5.2.2. Métodos	6
5.3. Armazém	6
5.3.1. Propriedades	6
5.3.2. Métodos	6
5.4. Funcionário	7
5.4.1. Propriedades	7
5.4.2. Métodos	7
5.5. Viatura	7
5.5.1. Propriedades	7
5.5.2. Métodos	7
5.6. Serviço	8
5.6.1. Propriedades	8
5.6.2. Métodos	8
5.7. Orçamento (Subclasse)	8
5.7.1. Propriedades	8
5.7.2. Métodos	8
5.8. Documento (Superclasse)	8
5.8.1. Propriedades Partilhadas	8
5.8.2. Métodos Partilhados	8

5.9. Empresa (Superclasse)	9
5.9.1. Propriedades Partilhadas	9
5.9.2. Métodos Partilhados	9
5.10. Organização (Subclasse)	9
5.10.1. Propriedades	9
5.10.2. Métodos	9
5.11. Subcontratado – (Subclasse)	10
5.11.1. Propriedades	10
5.11.2. Métodos	10
6. Análise e Desenho da Solução	10
6.1. Arquitetura do Sistema	10
6.2. Diagrama UML	11
6.3. Protótipo da Interface Gráfica (UI)	12
6.3.1. Ecrã do Login e Ecrã do Registo de Conta	12
6.3.2. Ecrã Principal do Programa	13
6.4. Persistência de Dados	13
6.5. Princípios da Programação Orientada a Objetos	14
6.6. Realização de Testes Unitários	14
Conclusão	14

Índice de Figuras

Figura I – Diagrama UML	11
Figura III - Mockup do Login	12
Figura II - Mockup do Registo de Conta	12
Figura IV - Mockup do Ecrã Principal	13

Índice de Tabelas

Tabela 1 - Requisitos Funcionais	5
Tabela 2 - Requisitos Não Funcionais	5

1. Introdução

No âmbito da unidade curricular *Programação Orientada a Objetos* lecionada no 1º semestre, o atual relatório visa apresentar a análise, o levantamento de requisitos e a implementação de uma solução orientada a objetos sobre a Gestão de uma Obra de Construção Civil. O projeto será desenvolvido na linguagem de programação *C#* e com interface gráfica (UI) desenvolvida em *WPF*.

2. Objetivo

Pretende-se conceber um sistema que responda a todas as necessidades básicas que implica uma Gestão de uma Obra de Construção Civil, aplicando os princípios fundamentais da *Programação Orientada a Objetos*, abstração, encapsulamento, herança e polimorfismo. Dentro das necessidades básicas encontra-se atividades como controlar e gerir os custos, recursos e atividades de determinadas obras.

3. Análise do problema

Atualmente, muitas empresas no setor da construção civil encontram dificuldades na gestão das suas obras e projetos. Nas dificuldades mais comuns, encontram-se: controlo de custo, gestão dos materiais e acompanhamento de mão obra, quer seja própria ou subcontratada, pois muitos destes processos são realizados manualmente, podendo levar a perdas de informação, cálculos mal efetuados dos custos, entre outros problemas. Pretende-se centralizar todas as soluções das dificuldades num único programa capaz de proporcionar uma melhor e mais eficiente gestão, automatizando processos para que assim se evite possíveis perdas de informação e prováveis erros de cálculos.

4. Levantamento de Requisitos

Para um bom desenvolvimento e uma boa implementação do programa, é de boa prática realizar um levantamento de requisitos, isto é, realizar o processo de identificar e documentar as necessidades e expectativas dos utilizadores e/ou dos *stakeholders*.

4.1. Requisitos Funcionais

Os requisitos funcionais descrevem o que o programa deve fazer, ou seja, as suas funções e o comportamento esperado.

Código	Descrição
RF01	Criar e editar obra
RF02	Registar material e o seu custo unitário
RF03	Gerir stocks por armazém
RF04	Registar funcionários e subcontratados
RF05	Atribuir serviços a obras
RF06	Registar viaturas

RF07	Calcular custos totais (material + mão de obra)
RF08	Gerar orçamentos e documentos
RF09	Guardar e carregar dados persistentes
RF10	Apresentar dados numa interface gráfica (UI)

Tabela 1 - Requisitos Funcionais

4.2. Requisitos Não Funcionais

Os requisitos não funcionais definem como o programa deve operar, incluindo segurança, desempenho, usabilidade, fiabilidade e escalabilidade.

Código	Descrição
RNF01	Linguagem: C#
RNF02	Interface: WPF
RNF03	Paradigma: Orientado a objetos
RNF04	Persistência: CSV ou JSON/XML
RNF05	Boas práticas: Uso de MVVM e de classes

Tabela 2 - Requisitos Não Funcionais

5. Identificação das Principais Classes

As principais classes do programa foram identificadas com base nos requisitos funcionais, representando as entidades e as operações essenciais à gestão de obras. Esta estrutura facilita a organização e a reutilização do código, bem como a sua manutenção. Assim, garante-se a clara divisão de responsabilidades e a boa implementação seguindo as regras de negócio.

5.1. Obra

A classe Obra representa os empreendimentos desenvolvidos pela empresa. Reúne toda a informação associada à execução de uma determinada obra, tal como, os recursos materiais, humanos e logísticos envolvidos. Esta classe permite o acompanhamento e controlo das várias fases do empreendimento.

5.1.1. Propriedades

- ID: Identificador único;
- Nome Cliente: Nome do cliente da obra;
- Morada: Local onde se realiza a obra;
- Estado: Estado atual da obra (Acabada, em progresso, etc.);
- List<Material>: Lista de materiais usados na obra;
- List<Funcionários>: Lista de funcionários a trabalhar na obra;
- List<Serviços>: Serviços prestados na obra;
- Orçamento: Orçamento prestado para a obra.

5.1.2. Métodos

- Adicionar Material: Adiciona material à obra;
- Adicionar Funcionário: Adiciona funcionário à obra;
- Remover Funcionário: Remove funcionário da obra;
- Adicionar Serviço: Adiciona um serviço à obra;
- Atualizar Estado: Atualiza o estado da obra;
- Calcular Custo: Calcula custo associado à obra.

5.2. Material

A classe Material descreve os recursos físicos utilizados nos empreendimentos da empresa, bem como os recursos armazenados em stock para futura utilização. É fundamental para a gestão de inventário e para o controlo de custos associados ao consumo de diferentes materiais ao longo dos projetos de empreendimento.

5.2.1. Propriedades

- ID: Identificador único do material;
- Nome: Nome do material;
- Quantidade: Quantidade de material;
- Custo Unitário: Custo do material por peça.

5.2.2. Métodos

- Atualizar Custo Unitário: Atualiza o custo unitário do material;
- Ver Descrição: Ver a descrição do material.

5.3. Armazém

A classe Armazém é responsável pela gestão e controlo do stock de materiais existentes na empresa, permitindo organizar e monitorizar os recursos físicos disponíveis.

5.3.1. Propriedades

- ID: Identificador único de armazém;
- Localização: Morada do armazém;
- List<Material>: Lista de material em stock no armazém;
- Servente: Funcionário que gere o stock do armazém.

5.3.2. Métodos

- Adicionar Material: Adiciona stock de um tipo de material ao armazém;
- Remover Material: Remove stock de um tipo de material ao armazém;
- Adicionar Servente: Adiciona um funcionário como servente de armazém;
- Remover Servente: Remove um funcionário como servente de armazém;
- Ver Stock: Visualizar stock disponível;
- Valor Stock: Calcular o valor total do stock do armazém.

5.4. Funcionário

A classe Funcionário representa os colaboradores da empresa. Contém informação necessária à gestão de trabalhadores, permitindo o acompanhamento das suas atividades e custo de trabalho.

5.4.1. Propriedades

- ID: Identificador único de funcionário;
- Nome: Nome completo do funcionário;
- NIF: Número Identificação Fiscal do funcionário;
- Telefone: Número de telefone do funcionário;
- Email: Email do funcionário;
- Cargo: Cargo desempenhado do funcionário;
- Salário Hora: Valor recebido por hora do funcionário;
- Horas Trabalhadas: Número de horas trabalhadas pelo funcionário;
- Data Início: Data que o funcionário começou a trabalhar na organização.

5.4.2. Métodos

- Atualizar Cargo: Mudar cargo do funcionário;
- Calcular Salário: Calcula o salário do funcionário;
- Aumentar Salário: Aumenta salário do funcionário percentualmente;
- Adicionar Horas: Atualiza o número de horas trabalhadas pelo funcionário;
- Mostrar Informação: Mostrar informações do funcionário.

5.5. Viatura

A classe Viatura modela os veículos pertencentes à empresa. É responsável pela gestão da frota, assegurando o registo e controlo da sua manutenção.

5.5.1. Propriedades

- VIN: VIN da viatura;
- Marca: Marca da viatura;
- Modelo: Modelo da viatura;
- Cor: Cor da viatura;
- Data Última Inspeção: Data da última inspeção efetuada;
- Data Última Revisão: Data da última revisão efetuada;
- Km Última Revisão: Número de quilómetros na última revisão.

5.5.2. Métodos

- Verificar Inspeção: Permite ver se tem a inspeção atualizada e qual a data limite da próxima inspeção;
- Verificar Revisão: Permite ver a que quilómetros e data tem de efetuar a próxima inspeção.

5.6. Serviço

A classe Serviço representa atividades ou tarefas executadas por empresas subcontratadas nos empreendimentos da empresa que opera o software. Permite o registo e o acompanhamento do estado do serviço prestado.

5.6.1. Propriedades

- ID: Identificador único do Serviço;
- Empresa: Subcontratado que efetua o serviço;
- Estado: Estado do serviço (Terminado, em progresso, etc.);
- Data Início: Data de Início do serviço;
- Data Fim: Data de fim do serviço (só terá valor aquando terminada o serviço);
- Horas Serviço: Número de horas prestadas no serviço.

5.6.2. Métodos

- Duração: Tempo, em dias, da duração do serviço;
- Atualizar Estado: Atualizar o estado do serviço;
- Custo Total: Custo total do serviço.

5.7. Orçamento (Subclasse)

A classe Orçamento reflete o planeamento e o controlo financeiro das obras. É utilizada para estimar os possíveis custos de um empreendimento.

5.7.1. Propriedades

- ID: Identificador único do Orçamento;
- Obra: Obra a que o Orçamento diz respeito;
- Custo Total: Custo total previsto para a obra a que o Orçamento diz respeito.

5.7.2. Métodos

- Atualizar Valor: Atualiza o valor total do Orçamento;

5.8. Documento (Superclasse)

A classe Documento representa registos formais e administrativos produzidos no âmbito das atividades da empresa. Pode incluir orçamentos, relatórios ou outros tipos de documentos que sirvam de suporte à gestão da empresa.

5.8.1. Propriedades Partilhadas

- Tipo Documento: Identificação do tipo de Documento;
- Data Emissão: Data de emissão do Documento;

5.8.2. Métodos Partilhados

- Ver Resumo: Ver resumo e dados do Documento em questão.

5.9. Empresa (Superclasse)

A classe Empresa representa a entidade base que caracteriza uma organização. É responsável por armazenar informação geral da empresa, de forma a identificar uma entidade. Serve de estrutura fundamental para classes que partilham dados do mesmo tipo.

5.9.1. Propriedades Partilhadas

- Nome: Nome da Empresa;
- NIF: Número Identificação Fiscal da Empresa;
- Morada: Morada da sede da Empresa;
- Data Fundação: Data de Fundação da Empresa.

5.9.2. Métodos Partilhados

- Mostrar Informação: Mostra informações partilhadas da Empresa.

5.10. Organização (Subclasse)

A subclasse Organização é uma especialização da classe Empresa e é o núcleo da gestão do sistema. Centraliza a administração de todos os componentes da organização, tais como, obras, viaturas, armazéns, funcionários, entre outros.

5.10.1. Propriedades

- CEO: Funcionário que gere a Organização;
- List<Obras>: Lista de obras que a Organização detém;
- List<Armazém>: Lista de armazéns detidos pela Organização;
- List<Funcionário>: Lista de funcionários que colaboram com a Organização;
- List<Viatura>: Lista da frota pertencente à Organização;
- List<Orçamento>: Lista de orçamentos referentes às obras da Organização;
- List<Subcontratados>: Lista de empresas subcontratadas pela Organização.

5.10.2. Métodos

- Adicionar Obra: Adiciona obra à lista detida pela Organização;
- Adicionar Armazém: Adiciona armazém à lista detida pela Organização;
- Remover Armazém: Remove armazém da lista detida pela Organização;
- Adicionar Viatura: Adiciona viatura à lista detida pela Organização;
- Remover Viatura: Remove viatura da lista detida pela Organização;
- Adicionar Funcionário: Adiciona funcionário à lista detida pela Organização;
- Remover Funcionário: Remove funcionário da lista detida pela Organização;
- Adicionar Subcontratado: Adiciona empresa subcontratada à lista detida pela Organização;
- Remover Subcontratado: Remove empresa subcontratada da lista detida pela Organização.

5.11. Subcontratado – (Subclasse)

A subclasse Subcontratado representa empresas externas à organização que prestam determinados serviços. A sua presença permite controlar o integrar os serviços subcontratados no planeamento dos empreendimentos.

5.11.1. Propriedades

- ID: Identificador único do subcontratado;
- Custo Hora: Custo por hora do subcontratado;
- Tipo Serviço: Tipo de Serviço que o subcontratado presta.

5.11.2. Métodos

- Atualizar Custo Hora: Atualiza o custo por hora do Subcontratado;

6. Análise e Desenho da Solução

A solução proposta foi desenvolvida com base nos princípios da Programação Orientada a Objetos, adotando uma estrutura modular e organizada de forma a promover a reutilização, a extensibilidade e a manutenção do código. O sistema foi concebido para garantir uma eficaz separação de responsabilidades entre as diferentes componentes da solução.

A arquitetura da solução seguiu o padrão MVVM (Model – View – ViewModel), uma abordagem amplamente usada em aplicações WPF, que permite uma ligação eficaz entre a interface gráfica e a lógica do código.

6.1. Arquitetura do Sistema

O sistema encontra-se estruturado e, consequentemente, dividido em três camadas principais:

- Camada de Apresentação (View) – responsável pela interface e desenvolvida em WPF, onde são apresentados dados e feitas as interações com o utilizador.
- Camada de Lógica de Negócio (ViewModel) – contém as regras de negócio, validações e controlo da aplicação. É nesta camada que se efetua cálculos e atualizações, bem como as interações entre a interface e os models.
- Camada de Dados (Model) – compreende as classes que representam as entidades, como, por exemplo, Obra, Material, Funcionário, etc. Compreende também a gestão e a persistência de dados através de ficheiros JSON.

Esta divisão permite garantir uma separação clara de responsabilidades, bem como uma arquitetura limpa e escalável.

6.2. Diagrama UML

O seguinte diagrama representa as propriedades e os métodos que as entidades possuem, bem como as ligações existentes entre elas.

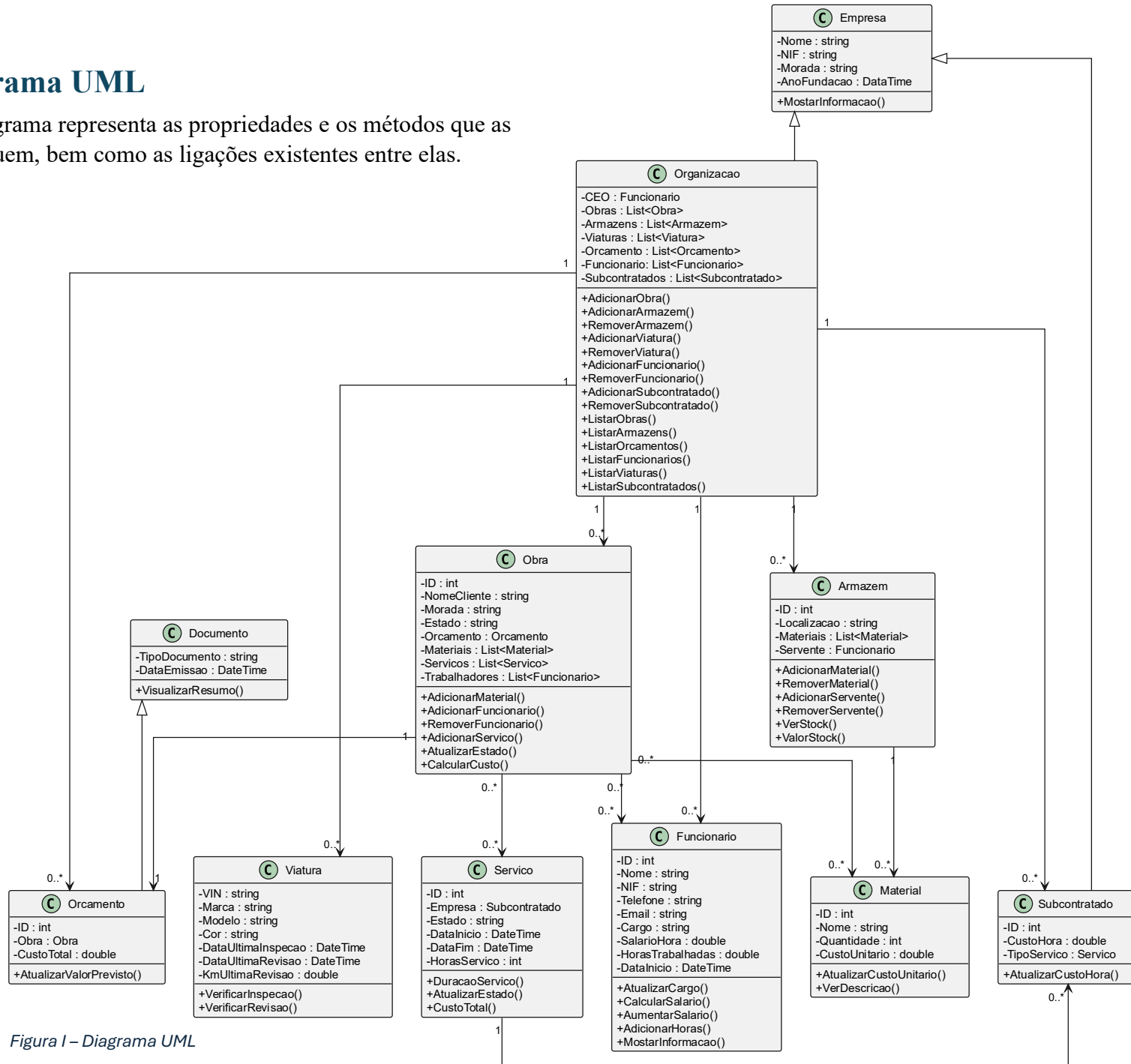


Figura 1 – Diagrama UML

6.3. Protótipo da Interface Gráfica (UI)

Esta secção apresenta os protótipos da interface gráfica da aplicação, desenvolvidos para ilustrar o design dos principais ecrãs do sistema. Os *mockups* foram criados de forma a representar a estrutura e as funcionalidades básicas da aplicação, respeitando o modelo MVVM adotado.

6.3.1. Ecrã do Login e Ecrã do Registo de Conta

O ecrã de login permite ao utilizador autenticar-se no sistema, inserindo as suas credenciais para aceder à aplicação. Caso não possuía uma conta, este ecrã ainda permite ao utilizador aceder ao ecrã de registo de conta.

Por sua vez, o ecrã de registo de conta possibilita o registo de novos utilizadores, permitindo criar contas com as credenciais necessárias para a utilização do sistema. Caso já possuía uma conta, este ecrã ainda permite ao utilizador aceder ao ecrã do login.



Figura III - Mockup do Registo de Conta

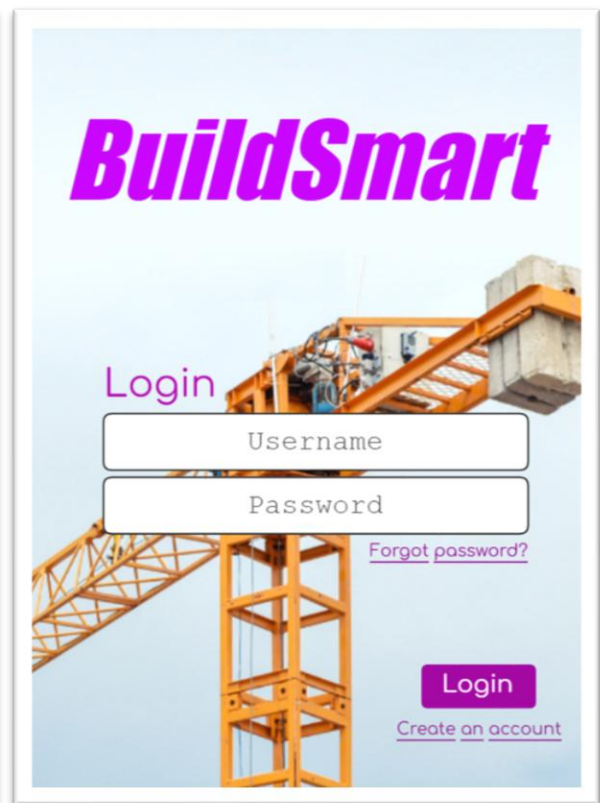


Figura II - Mockup do Login

6.3.2. Ecrã Principal do Programa

O ecrã principal apresenta a área de trabalho do sistema e constitui o ponto de acesso às principais funcionalidades da aplicação. Através deste ecrã, o utilizador pode aceder às diferentes utilidades oferecidas pelo programa, nomeadamente a gestão de obras, funcionários, veículos, viaturas, armazéns, etc., conforme as entidades previamente modeladas no diagrama UML.

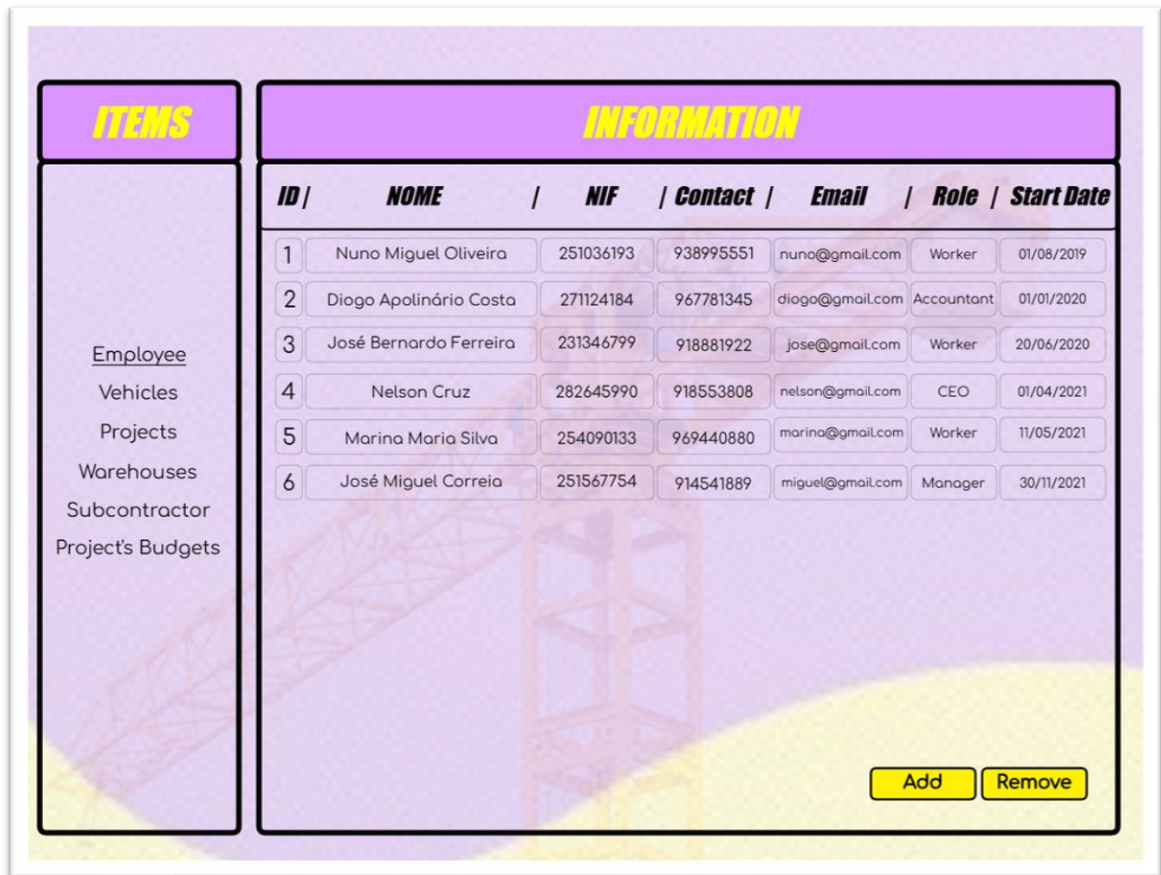


Figura IV - Mockup do Ecrã Principal

6.4. Persistência de Dados

A persistência de dados é necessária, uma vez que é imperativo que a informação seja guardada entre sessões. Esta vai ser efetuada através de ficheiros JSON, pois é um formato relativamente simples e compatível com C#, bem como a facilidade de serialização e desserialização dos objetos. O sistema irá permitir a gravação e o carregamento dos dados relativos a obras, funcionários, armazéns e as demais entidades, assegurando que a informação seja guardada entre sessões. O sistema também irá conter um mecanismo de verificação e tratamento de exceções para garantir a integridade dos dados durante as opções de leitura e escrita.

6.5. Princípios da Programação Orientada a Objetos

A aplicação foi desenhada em harmonia com os quatro pilares fundamentais da Programação Orientada a Objetos.

- **Abstração:** Consistiu na identificação e representação apenas das características e comportamentos essenciais de um conjunto de objetos, ignorando os detalhes específicos de cada um. Foi usado, por exemplo, na superclasse Empresa e nas subclasses Organização e Subcontratado.
- **Encapsulamento:** As propriedades das classes são protegidas através do *get* e do *set*, permitindo o controlo do acesso, sendo a própria classe a única que tem acesso às mesmas, garantindo a integridade da informação;
- **Herança:** Foi aplicada para promover a reutilização do código, nomeadamente entre a superclasses Empresa e as subclasses Organização e Subcontratado;
- **Polimorfismo:** Foi implementado através de redefinição de métodos, permitindo tratar diferentes tipo de objetos da mesma forma.

6.6. Realização de Testes Unitários

Durante a fase de implementação, serão realizados testes unitários com cobertura mínima de 50%. O objetivo dos testes unitários é garantir o correto funcionamento das funcionalidades essenciais do sistema. Estes irão incluir os métodos mais críticos das classes principais, como o cálculo de custos, a gestão de materiais, entre outras. A utilização de testes unitários irá permitir a validação da robustez do código, detetar eventuais anomalias e assegurar uma boa qualidade geral do sistema.

Conclusão

Nesta primeira fase do projeto, foi realizada a análise detalhada do problema, a definição dos requisitos e a idealização da arquitetura da solução. Foram também elaborados os protótipos da interface gráfica e o diagrama UML que suportará o desenvolvimento da aplicação.

A fase seguinte consistirá na implementação do sistema em linguagem C# com recurso ao *framework* WPF, seguindo o padrão de arquitetura MVVM descrito. Durante o desenvolvimento, irão ser aplicados os princípios da Programação Orientada a Objetos, bem com os testes unitários para garantir uma boa fiabilidade e qualidade da solução final.